

Project Report: Student Record Management System

1. Program Design

The Student Record Management System is a Python based application designed to manage student information using a combination of CSV and JSON files for persistent storage.

CSV Storage: Stores primary student attributes (reg_no, name, dateofbirth, address, contact, program, year_of_study) in a flat tabular format.

JSON Storage: Provides a secondary, structured storage of the same student data, indexed by registration number for fast lookup and flexibility.

System Architecture: Built using Object Oriented Programming (OOP) principles, the StudentManagementSystem class encapsulates all data persistence logic, while the main() function serves as the command line interface.

2. Key Functions

`add_student()`: Captures comprehensive student details, validates that the registration number is unique, and saves the data to both CSV and JSON formats.

`view_all_students()`: Aggregates data from storage to display a formatted table of all registered students, including their program and contact information.

`search_student()`: Performs a targeted search by registration number, retrieving and displaying all associated details from the merged data sources.

`update_student()`: Facilitates the modification of existing student records, allowing users to update specific fields while retaining current values where no new input is provided.

`delete_student()`: Safely removes a student's data from both the CSV file and the JSON object, ensuring data consistency.

3. Exception Handling Strategy

The system employs a multilayered approach to handle errors and ensure stability:

Custom Exceptions: Defines `StudentNotFoundError` and `DuplicateStudentError` to provide clear, domain specific feedback for operational failures.

Data Validation: Checks for empty registration numbers and duplicate entries before allowing data persistence.

Global Error Handling: Uses `try`, `except`, and `finally` blocks within the main loop to catch and log unexpected exceptions, preventing the application from crashing.

Logging: Maintains a detailed execution log in `student_system.log`, recording timestamps, system events (INFO), and error traces (WARNING/ERROR/CRITICAL) for debugging and auditing.

4. Testing Results

Success Case Adding Student: Successfully added student S005 ("Brighton Kwagala") and verified their presence in both `students.csv` and `students.json`.

Success Case Search: Searching for an existing registration number (e.g., S005) correctly retrieves full details including address and year of study.

Error Case Duplicate Reg No: Attempting to add a student with an existing ID (e.g., S005) is successfully blocked by the DuplicateStudentError handler.

Error Case Not Found: Searching for a non-existent ID correctly triggers a StudentNotFoundError message to the user.

File Resilience: The system automatically initializes students.csv and students.json with appropriate headers and structures if they are missing.

5. Conclusion

The application demonstrates proficiency in Python file handling, data management between different formats, and robust error management. It provides a reliable framework for managing student records with integrated logging for system monitoring.